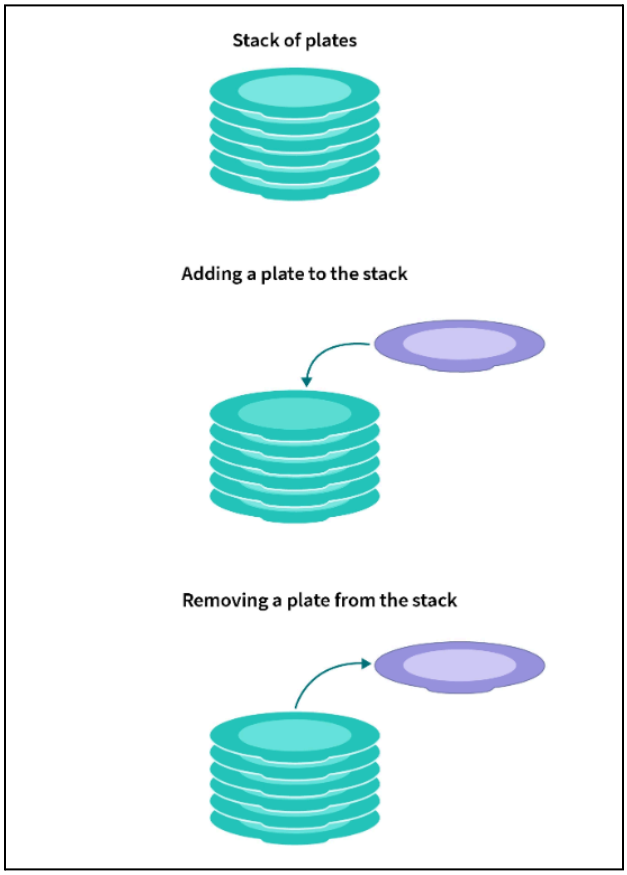
Stack

Like Array and Linked List, a stack is a simple data structure for storing data. In a stack, the order in which the data arrives is important. A pile of plates in a cafeteria is a good example of a stack. The plates are added to the stack as they are cleaned and placed on top. When a plate is required, it is taken from the top of the stack. The first plate placed on the stack is the last one to be used.

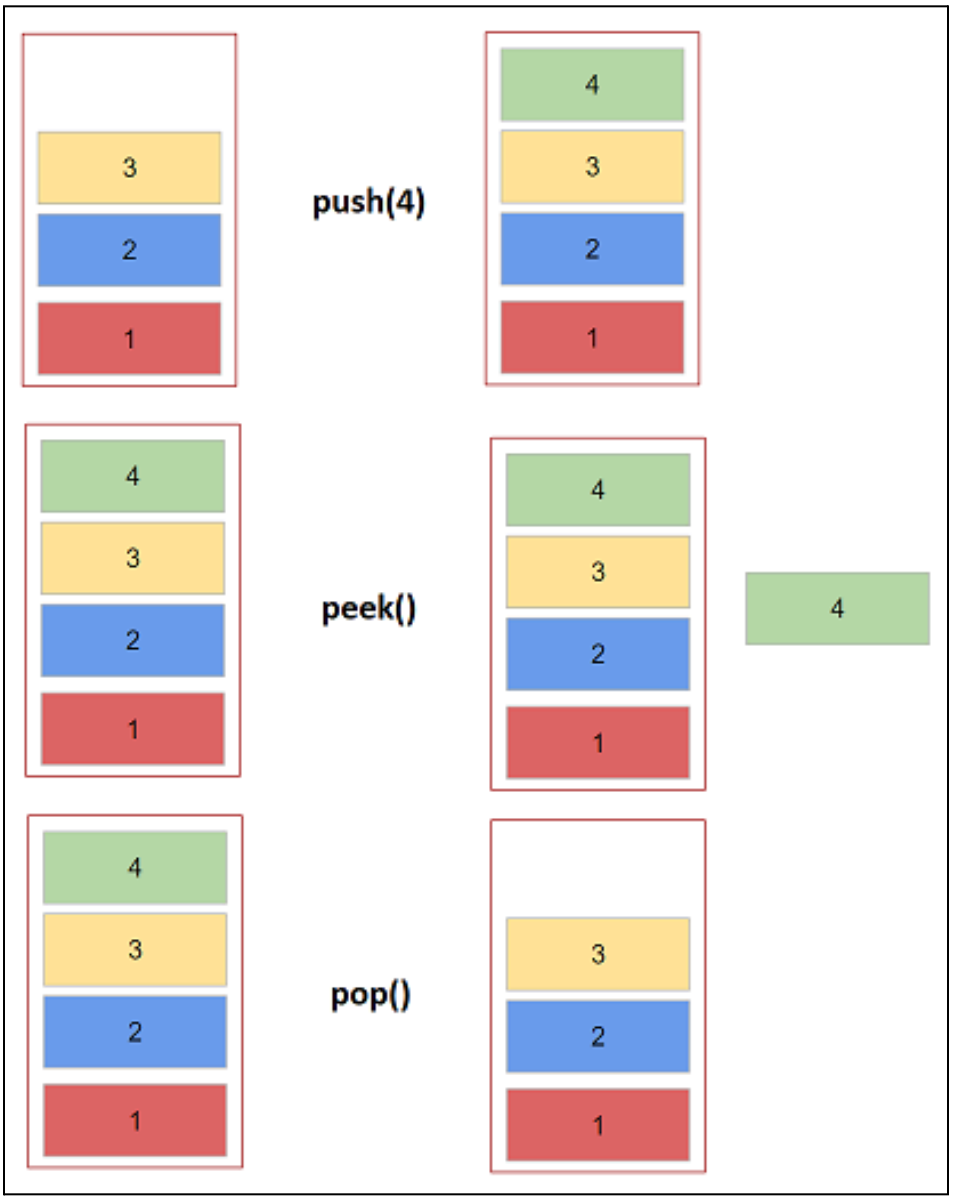


Definition: A stack is an ordered list in which insertion and deletion are done at one end, called the top. The last element inserted is the first one to be deleted. Hence, it is called the Last in First out (LIFO)

Hence, it is impossible to add or remove elements from the middle of the stack. The only element that can be taken out is the most recently added one.

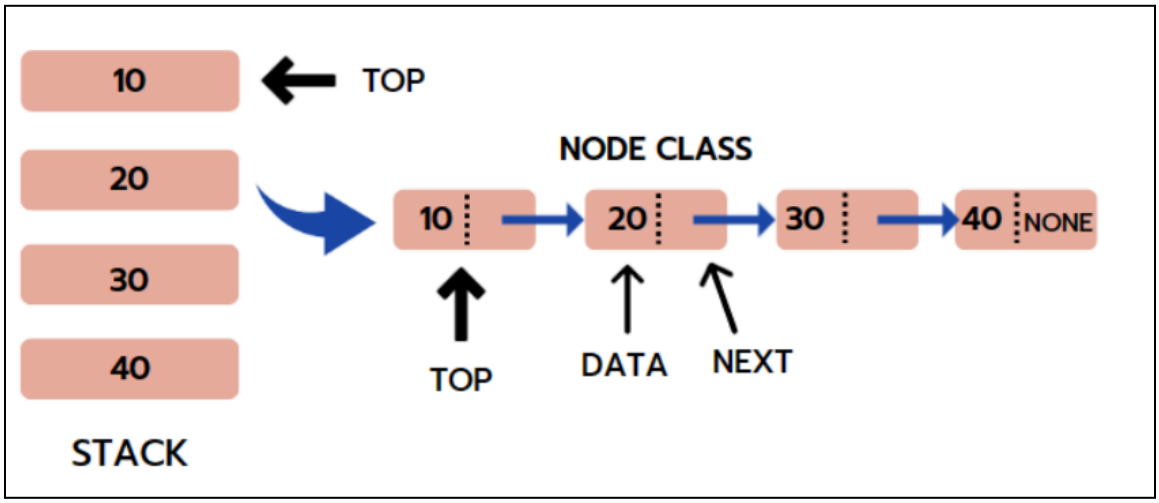
Stack Operations:

- push(obj): adds obj to the top of the stack ("overflow" error if the stack has fixed capacity, and is full)
- pop: removes and returns the item from the top of the stack ("underflow" error if the stack is empty)
- peek: returns the item that is on the top of the stack, but does not remove it



Stack Implementation:

- Linked List Based Stack:




```
class Node:
        function constructor(elem, next):
                self.elem = elem
                self.next = next
```

```
class Stack:
        function constructor():
                self.top = null
```

```
function push(elem):
        if top == null:
                top = new Node(elem, null)
        else:
                n = new Node(elem, null)
                n.next = top
                top = n
```

```
function pop():
        if top == null:
                print ("Stack Underflow")
        else:
                popped = top
                top = top.next
                popped.next = null
                return popped.elem
```

```
function peek():
        if top == null:
                print ("Stack is empty")
        else:
                return top.elem
```

```
function isEmpty():
        if top == null:
                return True
        else:
                return False
```

Stack Applications:

- Reverse:

```
1. begin with an empty stack and an input stream.
2. while there are more characters to read:
3.      read the next input character;
4.      push it onto the stack;
5. while the stack is not empty:
7.      c = pop the stack;
8.      print c;
```

```
function reverse(string):
        n = length of string
        stack = new Stack()
        for i = 0 to n - 1:
                stack.push(string[i])
        reversed_string = ""
        for i = 0 to n - 1:
                reversed_string = reversed_string + stack.pop()
```

- Expression Evaluation:

Let's first see what is infix, prefix, and postfix

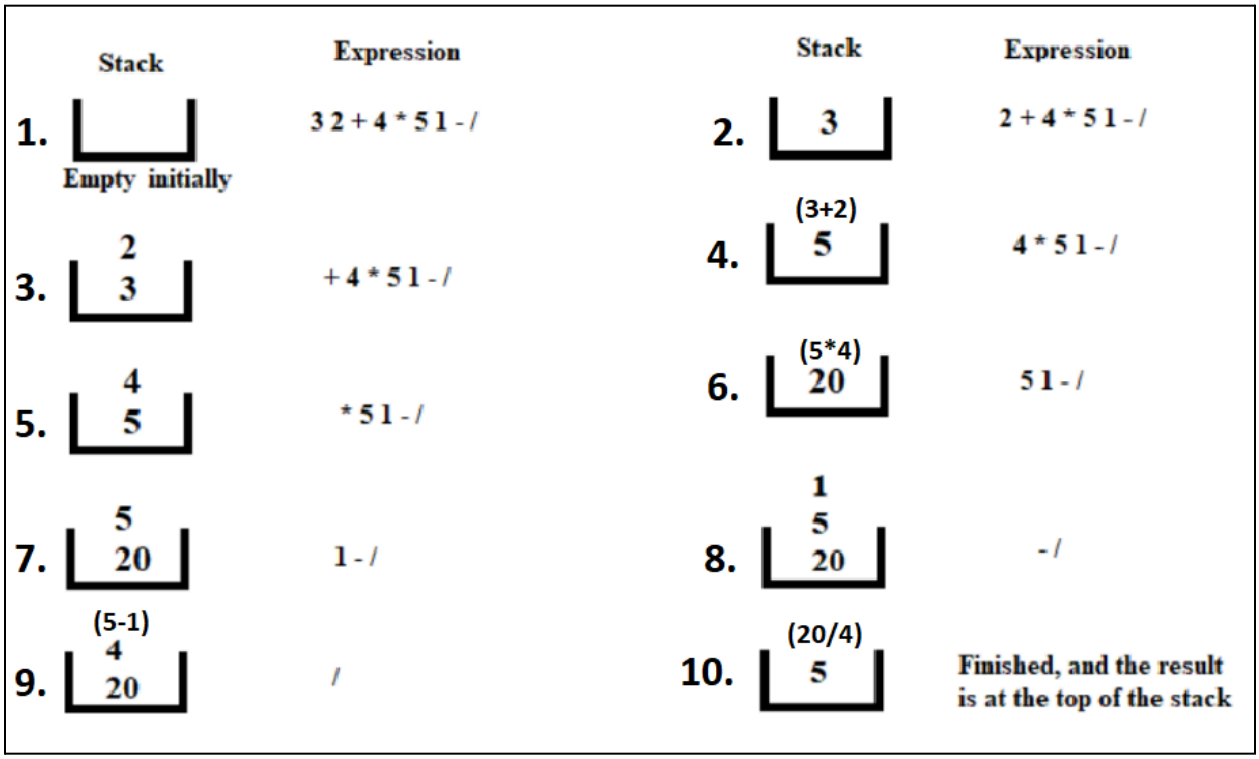
Infix: The operator is placed between the operands. This is the most common and familiar form to humans. E.g. A + B, (A + B) * C

Postfix: The operator is placed after the operands. E.g. A+B -> A B +, (A + B) * C ⇒ A B + C *

Prefix: The operator is placed before the operands. E.g. A+B -> + A B, (A + B) * C ⇒ * + A B C

In Expression Evaluation, when an arithmetic expression is presented in the postfix form, you can use a stack to evaluate it to get the final value. E.g.
((3 + 2) * 4) / (5 - 1) ⇒ 3 2 + 4 * 5 1 - / ⇒ 5

```
1. begin with an empty stack and an input stream (for the expression).
2. while there is more input to read:
3.      read the next input symbol;
4.      if it's an operand,
5.              then push it onto the stack;
6.      if it's an operator
7.              then pop two operands from the stack;
8.              operate on the operands;
9.              push the result;
10. pop the answer;
```



- Parentheses Matching

In many editors, it is common to want to know which left parenthesis (or brace or bracket, etc) will be matched when we type a right parenthesis. We want the most recent left parenthesis. Therefore, the editor can keep track of parentheses by pushing them onto a stack, and then popping them off as each right parenthesis is encountered.

We often have expressions involving "()[]{}" that require that the different types of parentheses are balanced. For example, the following are properly balanced:

(a (b + c) + d)
[(a b) (c d)]
([a {x y} b])

But the following are not:

(a (b + c) + d
[(a b] (c d))
([a {x y) b])

```
1.  begin with an empty stack and an input stream (for the expression).
2.  while there is more input to read:
3.      read the next input character;
4.      if it's an opening parenthesis/brace/bracket ("(" or "{" or "[")
5.              then push it onto the stack;
6.      if it's a closing parenthesis/brace/bracket (")" or "}" or "]")
7.              then pop the opening symbol from the stack;
8.              compare the closing with the opening symbol;
9.              if it matches
10.                     then continue with the next input character;
11.             if it does not match
12.                     then return false;
13. return true;
```